```python
#Mean
marks=[45,50,55,60,90]
mean=sum(marks)/len(marks)
print("Mean : ",mean)
```

```
Mean :  60.0
```

```python
#Median
import statistics
marks=[45,50,55,60,90]
median=statistics.median(marks)
print("Median : ",median)
```

```
Median :  55
```

```python
#Mode
import statistics
data=[10,20,20,30,40]
mode=statistics.mode(data)
print("Mode : ",mode)
```

```
Mode :  20
```

```python
import pandas as pd
df=pd.DataFrame({
    'Marks':[45,50,55,60,90]})
print("Mean : ")
print(df['Marks'].mean())
print("\nMedian : ")
print(df["Marks"].median())
print("\nMode : ")
print(df['Marks'].mode())
```

```
Mean :
60.0

Median :
55.0

Mode :
0    45
1    50
2    55
3    60
4    90
Name: Marks, dtype: int64
```

```python
#variance
import numpy as np
data=[10,12,14,16,18]
print(np.mean(data))
```

```
variance=np.var(data)
std=np.std(data)
print("Variance : ",variance)
print("Std : ",std)
```

```
14.0
Variance :  8.0
Std :  2.8284271247461903
```

```
#interquartile range(IQR)
IQR measures the spread of the middle 50% of data
It is less affected by outliers than variance and SD
Formula:
IQR=Q3-Q1
where
Q1=first quartile(25%)
Q2=third quartile(75%)


#IQR for outlier detection
lower limit=Q1-1.5*IQR
upper limit=Q3+1.5*IQR
any valu outside these limits is considered as outlier

#IQR
import numpy as np
data=[5,10,15,20,25,30,35,40]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
print("Q1 : ",Q1)
print("Q3 : ",Q3)
print("IQR : ",IQR)
```

```
Q1 :  13.75
Q3 :  31.25
IQR :  17.5
```

```
import numpy as np
data=[10,12,14,15,16,18,100]
Q1=np.percentile(data,25)
Q3=np.percentile(data,75)
IQR=Q3-Q1
lower=Q1-1.5*IQR
upper=Q3+1.5*IQR
print(lower)
print(upper)
outliers=[x for x in data if x<lower or x>upper]
print("Outliers: ",outliers)
```

```
7.0
23.0
Outliers:  [100]
```

```
#Probability Basics
probability is the measure of how likely an event is to occur
0=Impossible Event
1=Certain Event
Example
probability of getting head in a coin toss=0.5
probability of rolling a 6 on a dice=1/6

#1.Sample Space(S)
the set of all possible outcomes of an experiment
#2.Event(A)
An event is a subset of the sample space
#3.Probability of an event P(A)
Example:
Find probability of getting an even number from a dice
sample_space=[1,2,3,4,5,6]
event_A=[2,4,6]
P_A=len(event_A)/len(sample_space)
print("P(A)=",P_A)
#4.Conditional probability(A|B)
probability of A occuring given that B has alreadu occured
#5.Bayes' theorem
used to calculate the probability of an event based on prior knowledge
Example:Disease Testing
suppose:
P_Disease=0.01
P_Positive_given_Disease=0.95
P_Positive=0.05

sample_space=[1,2,3,4,5,6]
event_A=[2,4,6]
print("Sample Space(S):",sample_space)
print("Event A:",event_A)
P_A=len(event_A)/len(sample_space)
print("\nProbability of Event A")
print("P(A)=",P_A)
```

```
Sample Space(S): [1, 2, 3, 4, 5, 6]
Event A: [2, 4, 6]

Probability of Event A
P(A)= 0.5
```

```
#Example
#Disease Prevalence=1%
```

```
#Test sensitivity=99%
#Positive test probability=5%
P_Disease=0.01
P_Positive_given_Disease=0.95
P_Positive=0.05
P_Disease_given_positive=(P_Positive_given_Disease*P_Disease)/P_Positi
ve
print("\nBayes' Theorem Example")
print("P(Disease)=",P_Disease)
print("P(Positive|Disease)=",P_Positive_given_Disease)
print("P(Positive)=",P_Positive)
print("P(Disease|Positive)=",round(P_Disease_given_positive,4))
print("Percentage=",round(P_Disease_given_positive*100,2),"%")
print(P_Disease_given_positive)
```

```
Bayes' Theorem Example
P(Disease)= 0.01
P(Positive|Disease)= 0.95
P(Positive)= 0.05
P(Disease|Positive)= 0.19
Percentage= 19.0 %
0.18999999999999997
```

```
#3.Conditional Probability
#P(A|B)
event_A={2,4,6}
event_B={4,5,6}  #Numbers greater than 3
intersection=event_A.intersection(event_B)
P_A_given_B=len(intersection)/len(event_B)
print("\nEvent B:",event_B)
print("A intersection B:",intersection)
print("P(A|B)=",P_A_given_B)
```

```
Event B: {4, 5, 6}
A intersection B: {4, 6}
P(A|B)= 0.6666666666666666
```

```
#Correlation
correlation measures the relationship between two variables

Correlatio



Pearson correlation
measures linear relationship between two numerical variables
when to use pearson
Numerical data
```

```
Linear relationship
Less outlier
Example
Height & Weight
Study Hours & Marks
Advertising & Sales
```

```
import pandas as pd
df=pd.DataFrame({
    "Hours":[1,2,3,4,5],
    "Marks":[20,40,50,70,90]})
corr=df["Hours"].corr(df["Marks"])
print("Pearson Correlation",corr)
#close to +1 so it is Strong positive relation
```

```
Pearson Correlation 0.9948497511671097
```

```
#Spearman Correlation
Measures rank-based (monotonic) relationships
Used when:
Data is not normally distributed
Outliers exist
Relationship is not perfectly linear
```

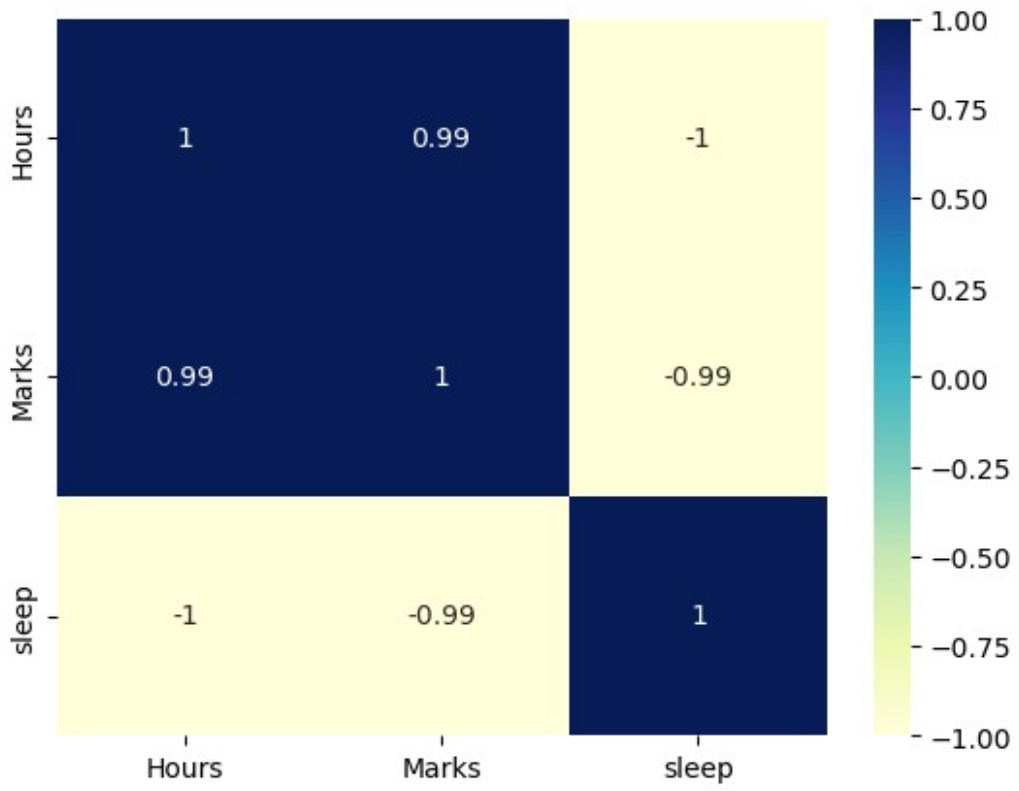
```
import pandas as pd
df=pd.DataFrame({
    "Rank_Study":[1,2,3,4,5],
    "Rank_Marks":[2,1,3,5,4]
})
corr=df["Rank_Study"].corr(
    df["Rank_Marks"],
    method="spearman")
print("Spearman Correlation:",corr)
```

```
Spearman Correlation: 0.7999999999999999
```

```
#correlation Matrix
import pandas as pd
df=pd.DataFrame({
    "Hours":[1,2,3,4,5],
    "Marks":[20,40,50,70,90],
    "sleep":[8,7,6,5,4]
})
print(df.corr())
```

```
         Hours    Marks    sleep
Hours  1.00000  0.99485 -1.00000
Marks  0.99485  1.00000 -0.99485
sleep -1.00000 -0.99485  1.00000
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=pd.DataFrame({
    "Hours":[1,2,3,4,5],
    "Marks":[20,40,50,70,90],
    "sleep":[8,7,6,5,4]
})
sns.heatmap(df.corr(),
            annot=True,
            cmap="YlGnBu")
plt.show()
#common cmap values
#cool warm
#viridis
#YlGnBu
#blues
#red
#greens
#magma
```



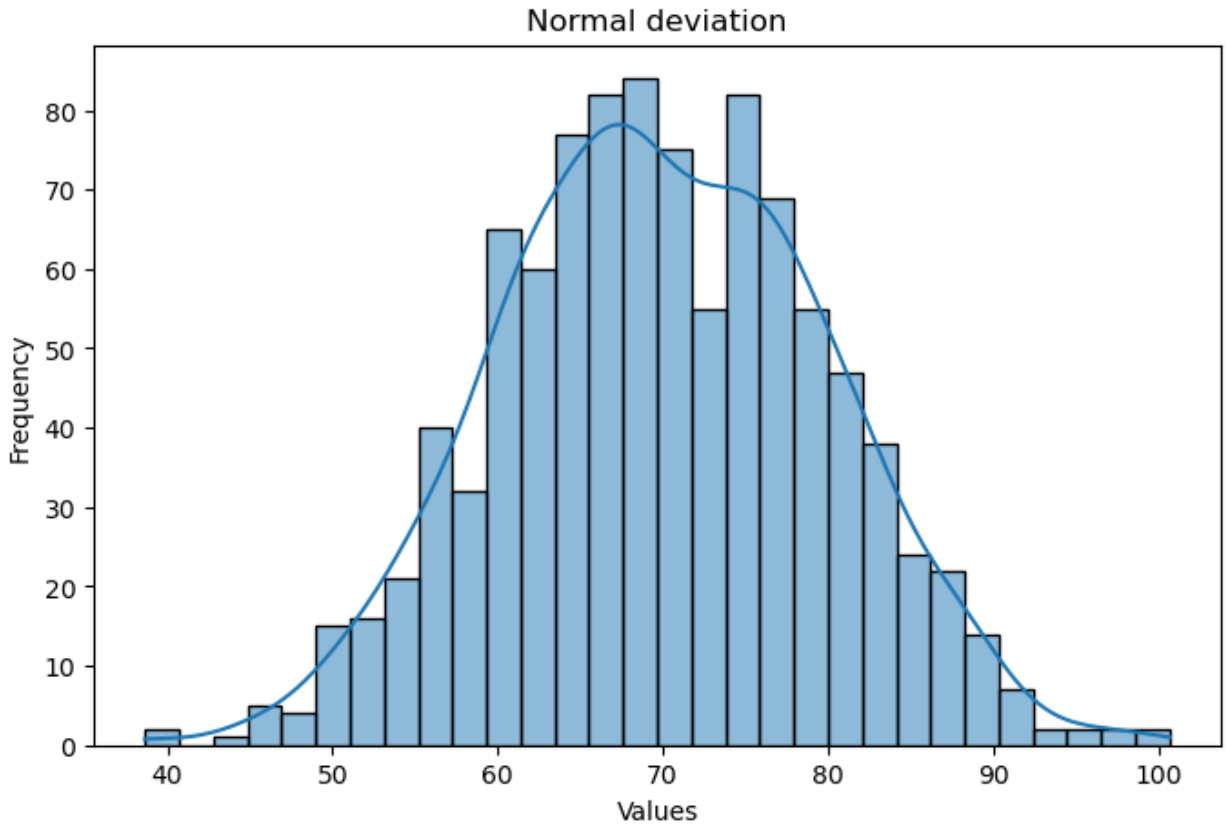
Causation causation means that one variable directly causes a change in another variable Example Study Hours Exam marks Studying more can directly improve exam performance this is an example of causation Normal Distribution: A normal distribution is a continous probability distribution

Right skew(positive skew) mean>median long tail on the right

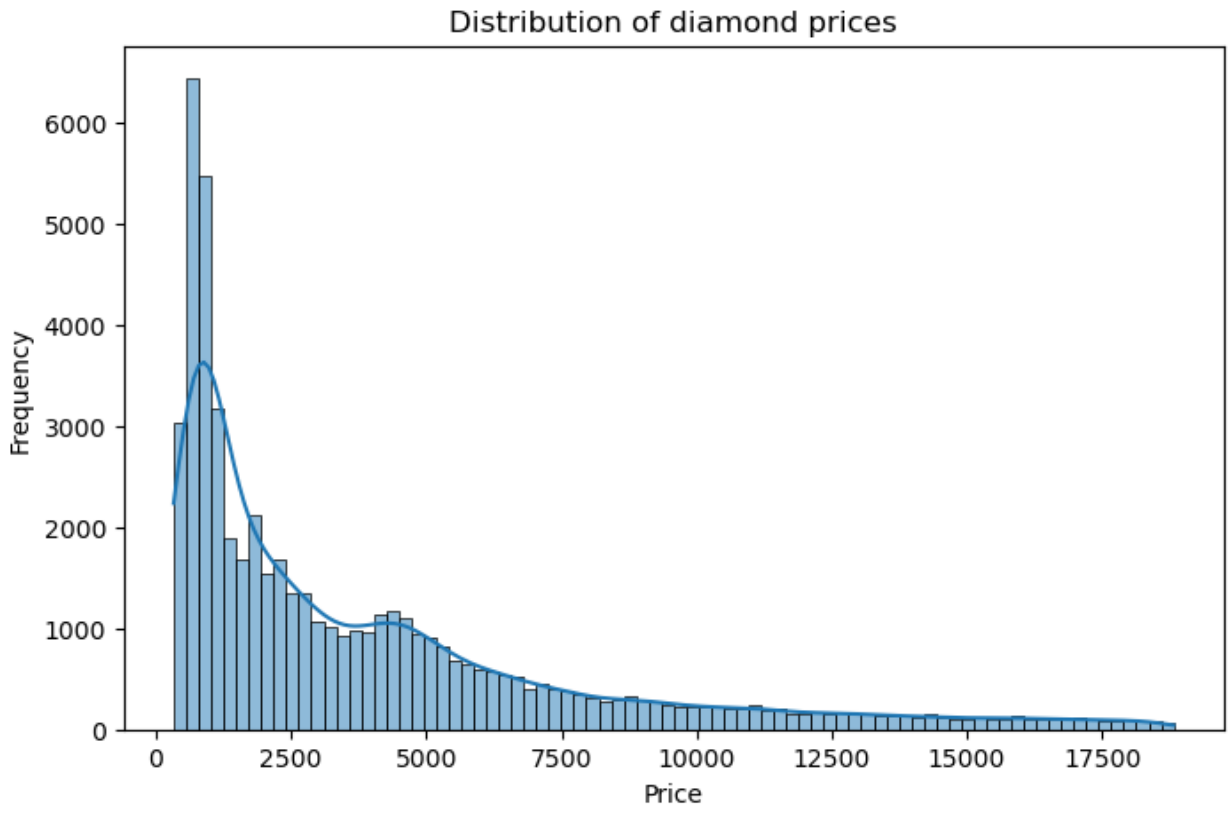
left skew(negative skew) mean<median long tail on the left

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
#generate normally distributed data
data=np.random.normal(
    loc=70,               #Mean
    scale=10,             #Standard deviation
    size=1000             #Number of values
)
#calculate mean and standard deviation
mean=np.mean(data)
std=np.std(data)
print("Mean : ",round(mean,2))
print("Standard Deviation : ",round(std,2))
#plot Histogram
plt.figure(figsize=(8,5))
sns.histplot(data,bins=30,kde=True)
plt.title("Normal deviation")
plt.xlabel("Values")
plt.ylabel("Frequency")
plt.show()
```

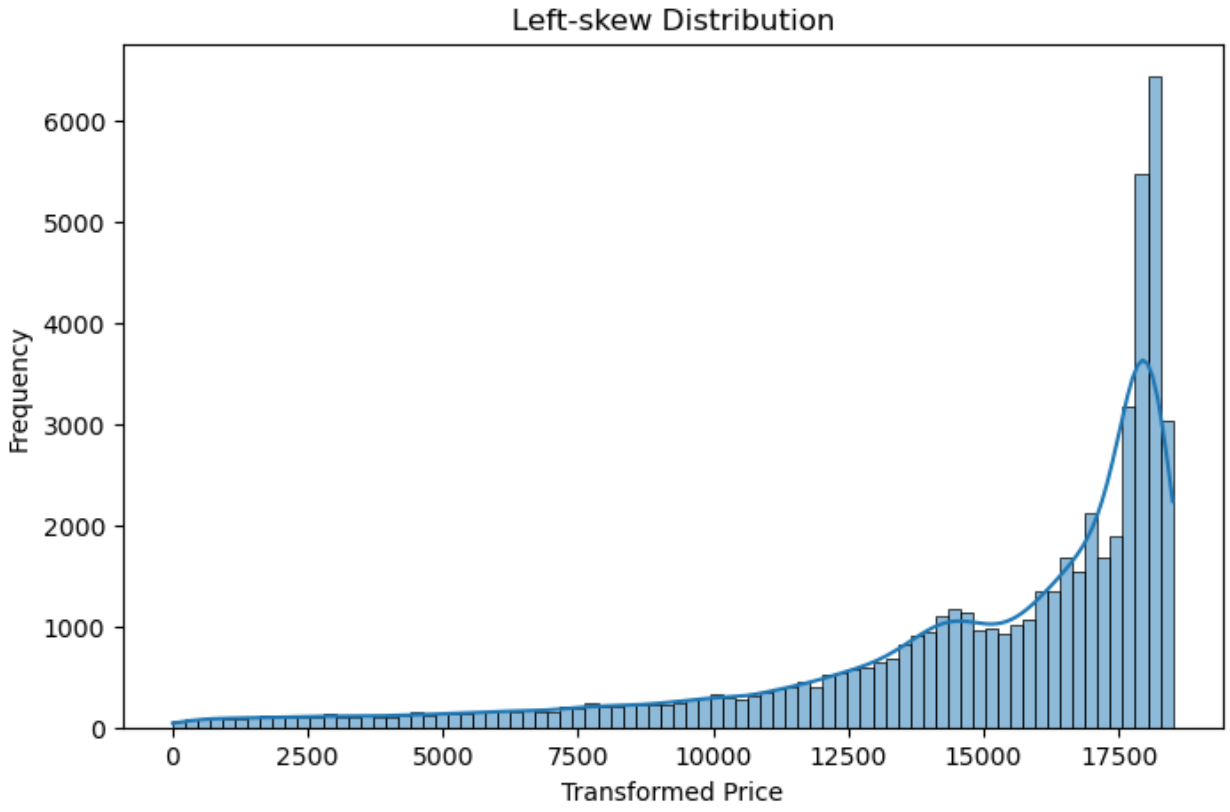
```
Mean :  69.92
Standard Deviation :  9.83
```




```
#RIGHT SKEW
import seaborn as sns
import matplotlib.pyplot as plt
#Load dataset
df=sns.load_dataset("diamonds")
#Histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(df["price"],kde=True)
plt.title("Distribution of diamond prices")
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.show()
```




```
#LEFT SKEW
import seaborn as sns
import matplotlib.pyplot as plt
#Load dataset
df=sns.load_dataset("diamonds")
#create left-skewed data
left_skew=df["price"].max()-df["price"]
#Histogram + KDE
plt.figure(figsize=(8,5))
sns.histplot(left_skew,kde=True)
plt.title("Left-skew Distribution")
plt.xlabel("Transformed Price")
plt.ylabel("Frequency")
plt.show()
#skewness
print("Skewness:",left_skew.skew())
```




```
Skewness: -1.6183952833835291
```

```
import numpy as np
scores=[60,65,70,75,80]
mean=np.mean(scores)
std=np.std(scores)
score=80
z_score=(score-mean)/std
print("mean : ",mean)
print("Standard Devaiation : ",std)
print("Z-Score : ",z_score)
```

```
mean :  70.0
Standard Devaiation :  7.0710678118654755
Z-Score :  1.414213562373095
```